



Universidad Autónoma Metropolitana

Actividades del curso de Temas Selectos II: Machine Learning

Alumno: Braulio Leonardo Santa Fe García

Profesor: Arturo Zuñiga Lopez

Fecha: 21/02/2025

Actividad 1: Expresión de w_1 en términos de la Covarianza y la Varianza

Una forma de estimar el error entre un modelo de regresión lineal y los valores observados es el error cuadrático medio (MSE), el cual se define como:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 x_i))^2$$

Las derivadas parciales del ECM con respecto a los parámetros w_0 y w_1 son las siguientes:

1. Derivada respecto a w_0 :

$$\frac{\partial}{\partial w_0} \left(\frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 x_i))^2 \right) = 0$$

Expandiendo y simplificando:

$$\sum_{i=1}^N (y_i - (w_0 + w_1 x_i)) = 0$$

2. Derivada respecto a w_1 :

$$\frac{\partial}{\partial w_1} \left(\frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 x_i))^2 \right) = 0$$

Expandiendo y simplificando:

$$\sum_{i=1}^N (y_i - (w_0 + w_1 x_i)) x_i = 0$$

Estas son las ecuaciones normales para los parámetros w_0 y w_1 , las cuales nos permiten encontrar los valores óptimos al resolver el sistema de ecuaciones.

El sistema de ecuaciones es:

$$\begin{bmatrix} N & \sum_{i=1}^N x_i \\ \sum_{i=1}^N x_i & \sum_{i=1}^N x_i^2 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^N y_i \\ \sum_{i=1}^N x_i y_i \end{bmatrix}$$

Para resolver este sistema usando la regla de Cramer, calculamos los determinantes de las matrices involucradas:

1. Determinante de A :

$$\det(A) = N \cdot \sum_{i=1}^N x_i^2 - \left(\sum_{i=1}^N x_i \right)^2$$

2. Determinante de A_0 (para w_0):

$$\det(A_0) = \sum_{i=1}^N y_i \cdot \sum_{i=1}^N x_i^2 - \left(\sum_{i=1}^N x_i \right) \cdot \sum_{i=1}^N x_i y_i$$

3. Determinante de A_1 (para w_1):

$$\det(A_1) = N \cdot \sum_{i=1}^N x_i y_i - \left(\sum_{i=1}^N x_i \right) \cdot \sum_{i=1}^N y_i$$

Finalmente, las soluciones son:

$$w_0 = \frac{\det(A_0)}{\det(A)}$$

$$w_1 = \frac{\det(A_1)}{\det(A)}$$

Las soluciones para los parámetros w_0 y w_1 son:

$$w_0 = \frac{\left(\sum_{i=1}^N y_i \right) \left(\sum_{i=1}^N x_i^2 \right) - \left(\sum_{i=1}^N x_i \right) \left(\sum_{i=1}^N x_i y_i \right)}{N \sum_{i=1}^N x_i^2 - \left(\sum_{i=1}^N x_i \right)^2}$$

$$w_1 = \frac{N \sum_{i=1}^N x_i y_i - \left(\sum_{i=1}^N x_i \right) \left(\sum_{i=1}^N y_i \right)}{N \sum_{i=1}^N x_i^2 - \left(\sum_{i=1}^N x_i \right)^2}$$

A continuación vamos a realizar la reducción de la expresión de w_1 para poderlo expresar en términos de la Covarianza y de la Varianza.

Consideremos el numerador de la expresión:

$$N \sum_{i=1}^N x_i y_i - \left(\sum_{i=1}^N x_i \right) \left(\sum_{i=1}^N y_i \right)$$

Queremos llegar a la siguiente forma:

$$\sum_{i=1}^N (x_i - \bar{x}) y_i$$

Donde $\bar{x}$ es la media de los x_i :

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

Expandimos la expresión $\sum_{i=1}^N (x_i - \bar{x}) y_i$:

$$\sum_{i=1}^N (x_i - \bar{x}) y_i = \sum_{i=1}^N x_i y_i - \sum_{i=1}^N \bar{x} y_i$$

Sustituimos $\bar{x}$:

$$\begin{aligned} \sum_{i=1}^N (x_i - \bar{x}) y_i &= \sum_{i=1}^N x_i y_i - \sum_{i=1}^N \left(\frac{1}{N} \sum_{i=1}^N x_i \right) y_i \\ &= \sum_{i=1}^N x_i y_i - \frac{1}{N} \left(\sum_{i=1}^N x_i \right) \left(\sum_{i=1}^N y_i \right) \end{aligned}$$

Por lo tanto:

$$N \sum_{i=1}^N x_i y_i - \left(\sum_{i=1}^N x_i \right) \left(\sum_{i=1}^N y_i \right) = N \sum_{i=1}^N (x_i - \bar{x}) y_i$$

Así, llegamos a la expresión para obtener la covarianza:

$$NCov(x, y) = N \sum_{i=1}^N (x_i - \bar{x}) y_i = N \sum_{i=1}^N x_i y_i - \left(\sum_{i=1}^N x_i \right) \left(\sum_{i=1}^N y_i \right)$$

A continuación, estudiaremos la reducción del denominador para llegar a la expresión clásica de la varianza.

Consideremos el denominador de w_1 :

$$N \sum_{i=1}^N x_i^2 - \left(\sum_{i=1}^N x_i \right)^2$$

Queremos expresar esto en términos de la varianza de x . La varianza de x , $\text{Var}(x)$, se define como:

$$\text{Var}(x) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

Expandimos esta expresión:

$$\text{Var}(x) = \frac{1}{N} \sum_{i=1}^N (x_i^2 - 2x_i\bar{x} + \bar{x}^2)$$

Podemos separar las sumas:

$$\text{Var}(x) = \frac{1}{N} \left(\sum_{i=1}^N x_i^2 - 2\bar{x} \sum_{i=1}^N x_i + \sum_{i=1}^N \bar{x}^2 \right)$$

Sabemos que $\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$, y también que la suma de N elementos iguales a $\bar{x}$ es $N\bar{x}$:

$$\text{Var}(x) = \frac{1}{N} \left(\sum_{i=1}^N x_i^2 - 2N\bar{x}^2 + N\bar{x}^2 \right)$$

Como $\sum_{i=1}^N x_i = N\bar{x}$:

$$\text{Var}(x) = \frac{1}{N} \left(\sum_{i=1}^N x_i^2 - N\bar{x}^2 \right)$$

Multiplicamos por N en ambos lados para relacionarlo con el denominador de w_1 :

$$N\text{Var}(x) = \sum_{i=1}^N x_i^2 - N\bar{x}^2$$

Y recordemos que $\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$:

$$N\text{Var}(x) = \sum_{i=1}^N x_i^2 - \frac{1}{N} \left(\sum_{i=1}^N x_i \right)^2$$

Multiplicamos nuevamente ambos términos por N :

$$N\text{Var}(x) = N \sum_{i=1}^N x_i^2 - \left(\sum_{i=1}^N x_i \right)^2$$

Por lo tanto, podemos ver que el denominador de w_1 es en realidad N veces la varianza de x :

$$N\text{Var}(x) = N \sum_{i=1}^N x_i^2 - \left(\sum_{i=1}^N x_i \right)^2$$

Finalmente tenemos que w_1 es:

$$w_1 = \frac{\text{Cov}(x, y)}{\text{Var}(x)}$$

Actividad 2. Coeficientes del modelo más óptimo en su forma matricial

Agrupamos todas las observaciones en una sola columna de tamaño $(n \times 1)$ y:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Agrupamos los coeficientes en un vector de tamaño (2×1) :

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix}$$

Para construir las observaciones de $\mathbf{x}$, necesitamos agregar una columna de 1's al inicio, con el fin de realizar correctamente la multiplicación $\mathbf{x}\boldsymbol{\beta}$.

$$\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}$$

La multiplicación sería:

$$\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} \beta_1 + x_1\beta_2 \\ \beta_1 + x_2\beta_2 \\ \vdots \\ \beta_1 + x_n\beta_2 \end{bmatrix}$$

La determinacion del error, en primera instancia es con la ecuación:

$$e(\beta) = \mathbf{y} - \mathbf{x}\beta \quad (1)$$

Si queremos calcular el error cuadrático medio MSE, utilizamos la siguiente expresión:

$$MSE = \frac{1}{N} \sum_{i=1}^N (e(\beta))^2$$

En su forma matricial sería:

$$MSE(\cdot) = \frac{1}{n} \mathbf{e}^T \mathbf{e}$$

Resolviendo con el valor de e:

$$MSE = \frac{1}{n} (\mathbf{y} - \mathbf{x}\beta)^T (\mathbf{y} - \mathbf{x}\beta)$$

$$MSE = \frac{1}{n} (\mathbf{y}^T - (\mathbf{x}\beta)^T) (\mathbf{y} - \mathbf{x}\beta)$$

$$= (\mathbf{y}^T - \beta^T \mathbf{x}^T) (\mathbf{y} - \mathbf{x}\beta) = \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{x}\beta - \beta^T \mathbf{x}^T \mathbf{y} + \beta^T \mathbf{x}^T \mathbf{x}\beta$$

Nota que el término $\mathbf{y}^T \mathbf{x}\beta$ es un escalar, por lo que se iguala a $\beta^T \mathbf{x}^T \mathbf{y}$. Por lo tanto, podemos simplificar la expresión a:

$$MSE = \frac{1}{n} (\mathbf{y}^T \mathbf{y} - 2\beta^T \mathbf{x}^T \mathbf{y} + \beta^T \mathbf{x}^T \mathbf{x}\beta)$$

Finalmente, tenemos la fórmula expandida del MSE:

$$MSE = \frac{1}{n} (\mathbf{y}^T \mathbf{y} - 2\beta^T \mathbf{x}^T \mathbf{y} + \beta^T \mathbf{x}^T \mathbf{x}\beta)$$

A continuación, buscamos el minimo de la función. Primeramente, encontramos el gradiente de la MSE con respecto a β , calculando la derivada de cada término con respecto a β .

$$\nabla_{\beta} MSE = \frac{1}{n} (-2\mathbf{x}^T \mathbf{y} + 2\mathbf{x}^T \mathbf{x}\beta)$$

Finalmente, podemos simplificar el gradiente:

$$\nabla_{\beta} MSE = \frac{2}{n} (\mathbf{x}^T \mathbf{x}\beta - \mathbf{x}^T \mathbf{y})$$

Para encontrar el valor de β que minimiza la MSE, igualamos el gradiente a cero:

$$\frac{2}{n} (\mathbf{x}^T \mathbf{x} \boldsymbol{\beta} - \mathbf{x}^T \mathbf{y}) = 0$$

Eliminamos el factor $\frac{2}{n}$ de ambos lados:

$$\mathbf{x}^T \mathbf{x} \boldsymbol{\beta} - \mathbf{x}^T \mathbf{y} = 0$$

Despejamos $\boldsymbol{\beta}$:

$$\mathbf{x}^T \mathbf{x} \boldsymbol{\beta} = \mathbf{x}^T \mathbf{y}$$

Multiplicamos ambos lados por $(\mathbf{x}^T \mathbf{x})^{-1}$ para obtener:

$$\boldsymbol{\beta} = (\mathbf{x}^T \mathbf{x})^{-1} \mathbf{x}^T \mathbf{y}$$

Este es el valor de $\boldsymbol{\beta}$ que minimiza la MSE.

Actividad 3. Gradiente Descendiente

El gradiente descendiente es un algoritmo de optimización utilizado para minimizar funciones de costo, comúnmente aplicado en el entrenamiento de modelos de aprendizaje automático. Su objetivo es encontrar los parámetros óptimos que minimizan una función de pérdida $\mathcal{L}(\theta)$, donde θ representa los parámetros del modelo (por ejemplo, pesos y sesgos).

La idea central es ajustar los parámetros en la dirección opuesta al gradiente de la función de costo, ya que este indica la dirección de máximo aumento. Por tanto, al moverse en la dirección negativa del gradiente, se reduce el valor de la función.

Fórmula general del gradiente descendiente

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

Donde:

- $\theta^{(t)}$ son los parámetros en la iteración t .
- η es la tasa de aprendizaje (learning rate), un hiperparámetro que controla el tamaño del paso.
- $\nabla_{\theta} \mathcal{L}(\theta)$ es el gradiente de la función de costo con respecto a los parámetros.

Este proceso se repite iterativamente hasta que se alcanza un mínimo (idealmente global, aunque en la práctica puede ser local).

La Figura 1 muestra la **superficie de costo** en un espacio tridimensional, junto con el camino que sigue el algoritmo desde un punto inicial hasta el mínimo. Se puede observar cómo los pasos del

gradiente descendiente recorren progresivamente la función de pérdida hasta alcanzar la zona de menor costo.

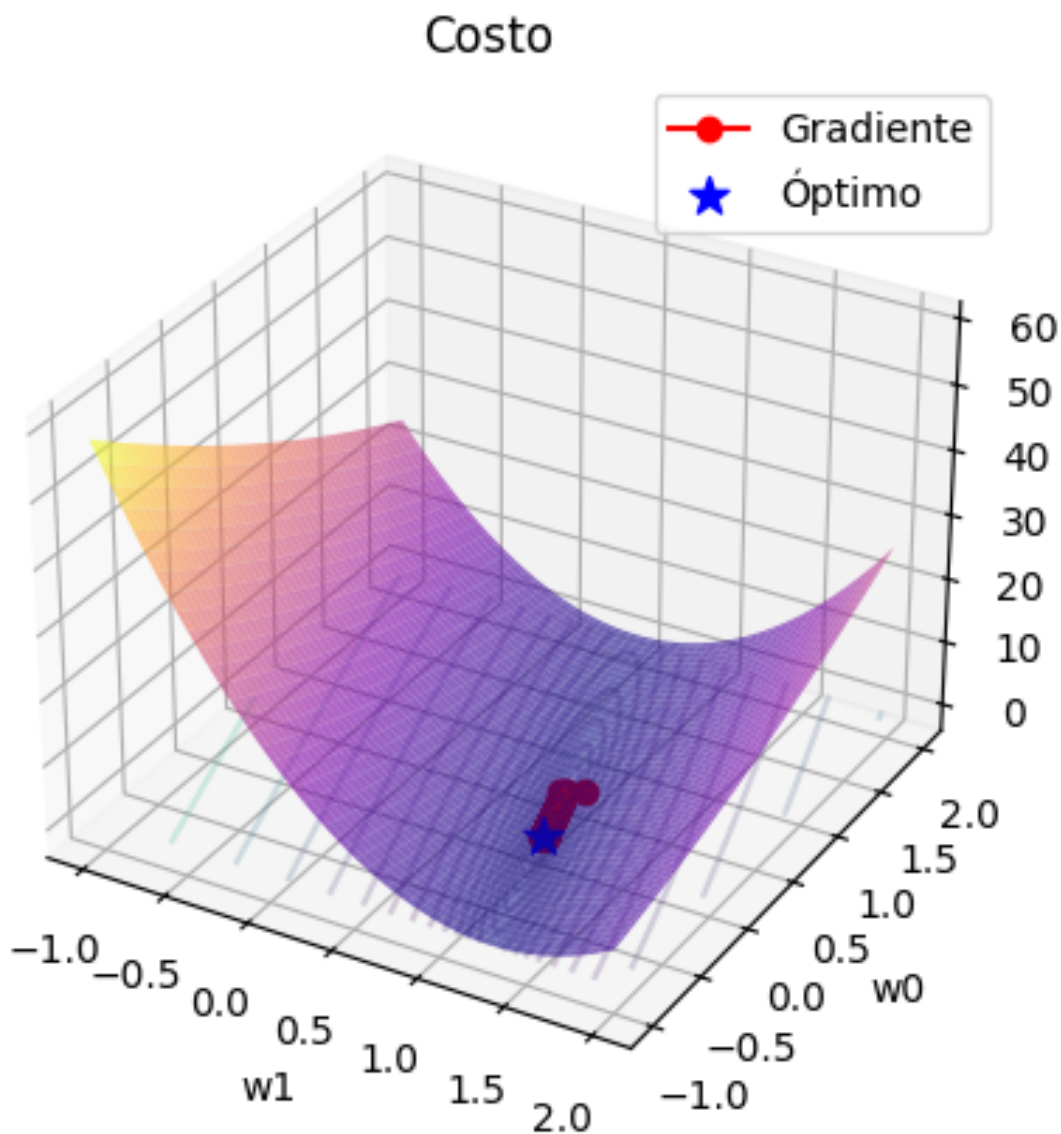


Figure 1: Superficie de costo y camino del gradiente descendiente

La Figura 2 presenta una **gráfica de contorno** de la misma función de costo, donde las líneas indican niveles de igual valor de la función. También aquí se puede observar claramente el camino del gradiente, el cual se dirige desde un punto inicial hacia el mínimo en el centro de las curvas de

nivel.

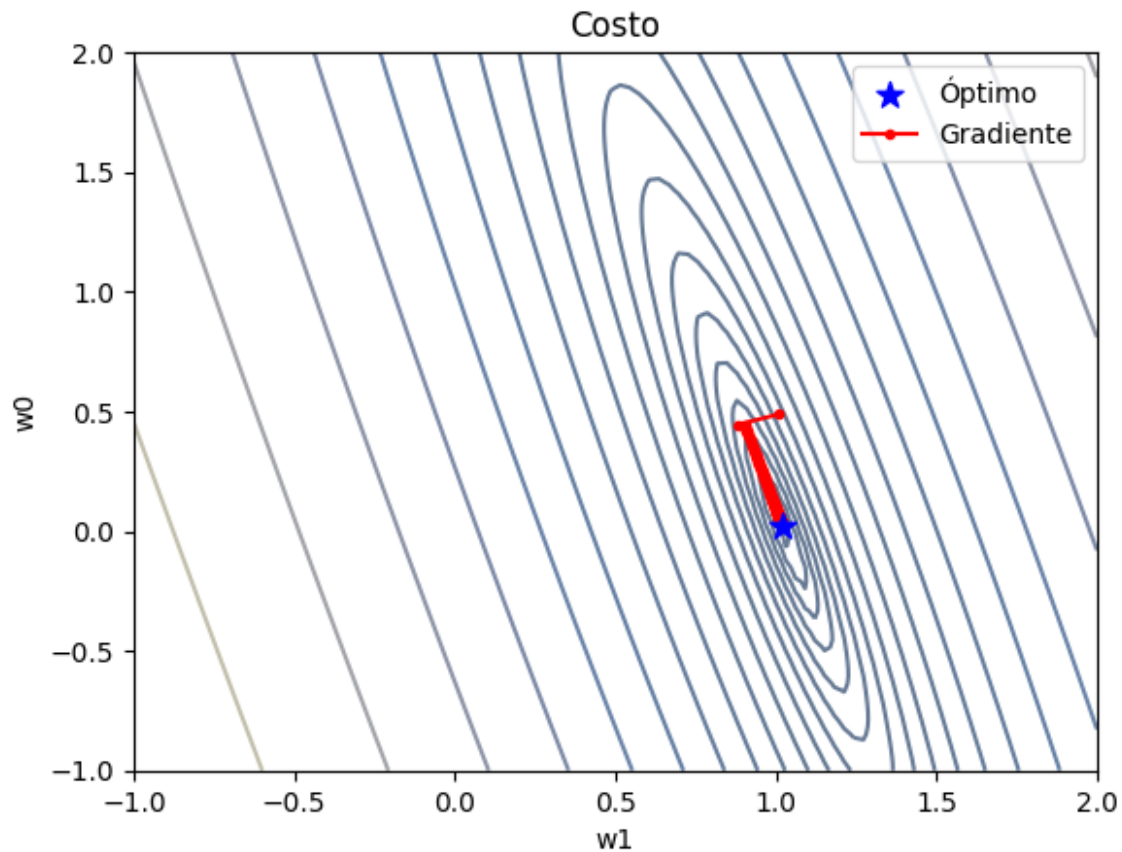


Figure 2: Gráfica de contorno del costo y camino del gradiente descendiente

Estas visualizaciones permiten comprender intuitivamente cómo el algoritmo ajusta los parámetros paso a paso hasta minimizar la función de pérdida durante el entrenamiento del modelo.

Actividad 4. Comparación de Regresión Lineal y Regresión RANSAC

La regresión lineal busca modelar la relación entre una variable independiente x y una dependiente y ajustando una línea recta de la forma:

$$y = wx + b$$

donde w es la pendiente y b es la intersección. El modelo se ajusta minimizando el error cuadrático medio:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2$$

Sin embargo, la regresión lineal tradicional es sensible a la presencia de valores atípicos (*outliers*), que pueden influir de forma significativa en la línea ajustada.

RANSAC (Random Sample Consensus) es un método alternativo robusto que selecciona iterativamente subconjuntos aleatorios de los datos y ajusta el modelo ignorando puntos que no se ajustan bien (considerados como outliers). De esta manera, se obtiene un modelo más representativo del comportamiento general de los datos.

Comparación:

- **Regresión Lineal:** eficiente y sencilla, pero vulnerable a valores atípicos.
- **RANSAC:** más robusta ante outliers, aunque implica mayor costo computacional.

En esta actividad, trabajamos con la base de datos *precios_casas.csv*, proporcionada por el profesor, la cual contiene diversas características de las viviendas. Nuestro objetivo principal fue identificar cuáles de estas características están más relacionadas con el precio de las propiedades. Para ello, llevamos a cabo un análisis de correlación considerando todas las variables, con especial atención al precio.

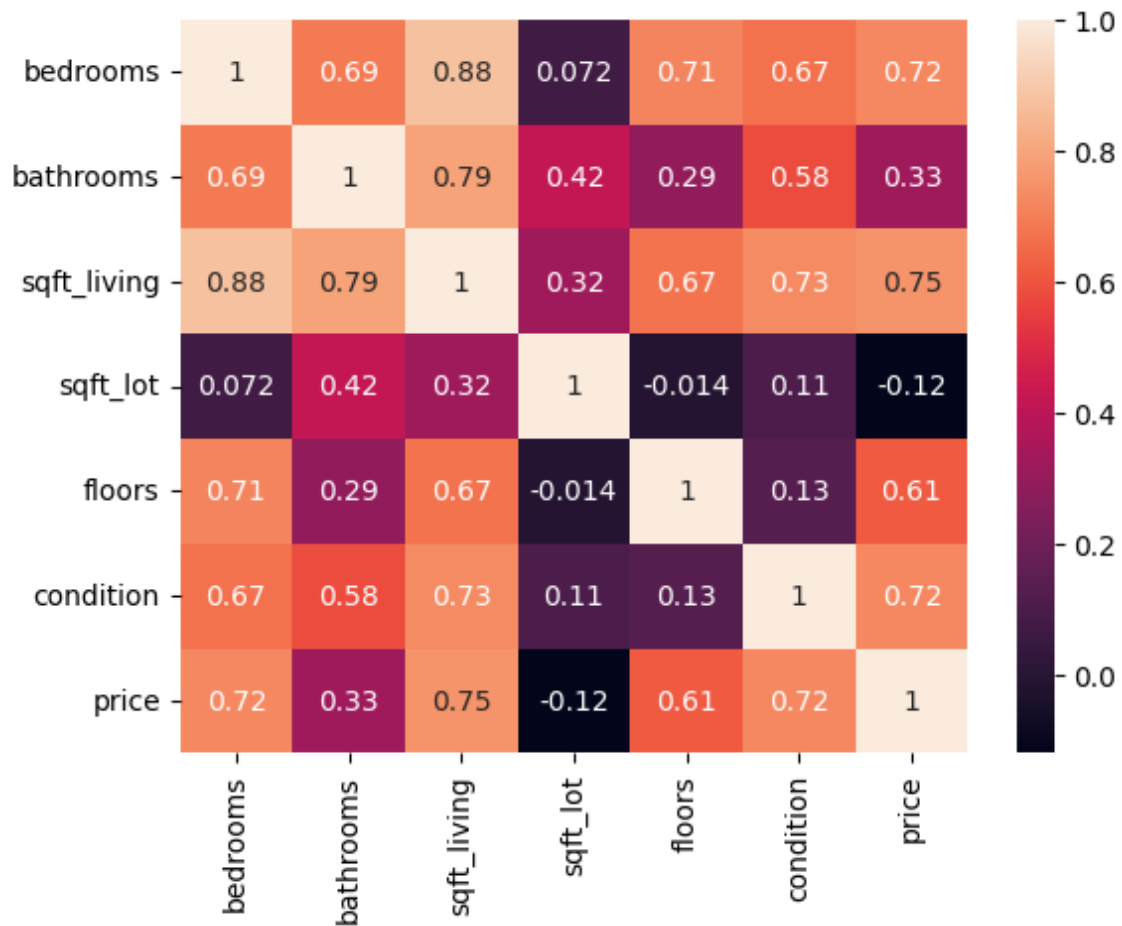


Figure 3: Matriz de correlación entre las variables de la base de datos.

La Figura 3 muestra la matriz de correlación de las características evaluadas.

Como se puede observar en la misma figura, la variable *sqft_living* es la que más se relaciona con el precio. Por esta razón, para generar predicciones sobre el precio, realizamos una regresión lineal utilizando estas dos variables. Cabe resaltar que se redujo el número de datos para mejorar la visualización.

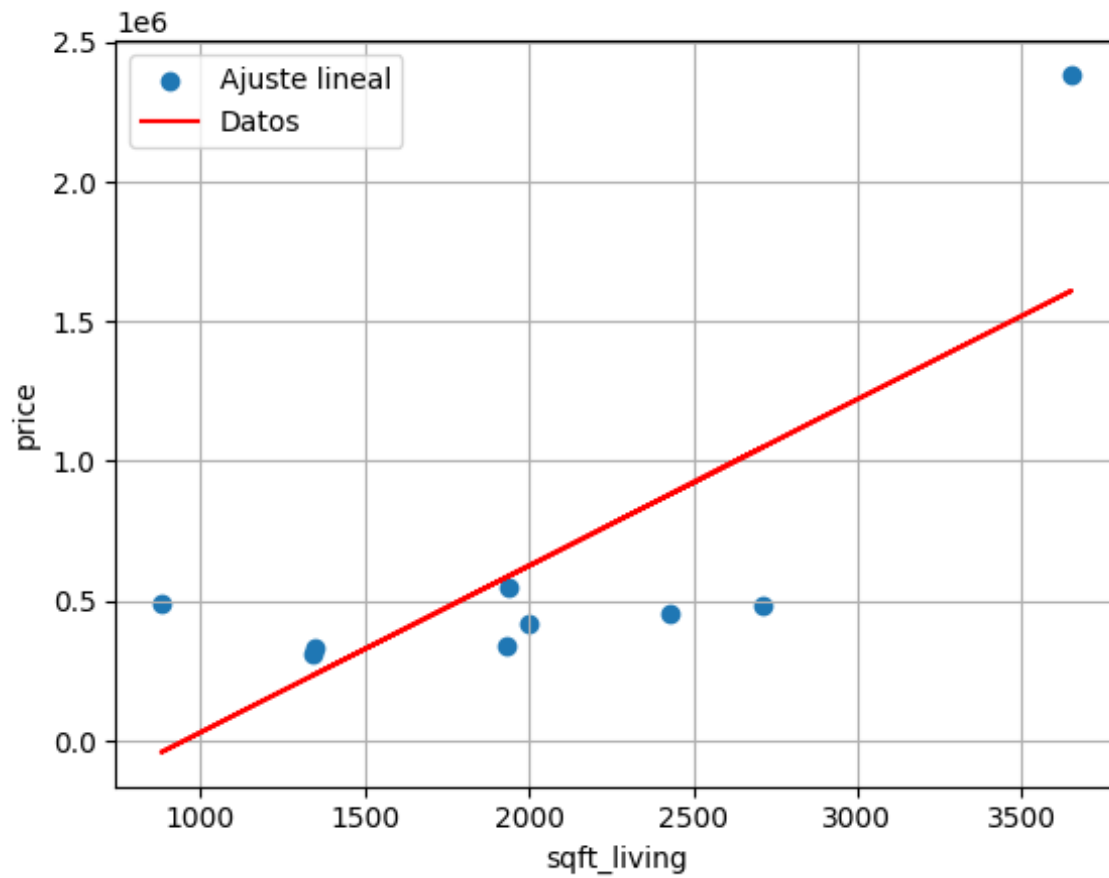


Figure 4: Regresión lineal entre *sqft_living* y el precio de las viviendas.

Como se observa en la Figura 4, la presencia de outliers influye considerablemente en la línea ajustada. Para mitigar este efecto, aplicamos una regresión RANSAC, la cual ofrece una solución más robusta al descartar estos puntos atípicos.

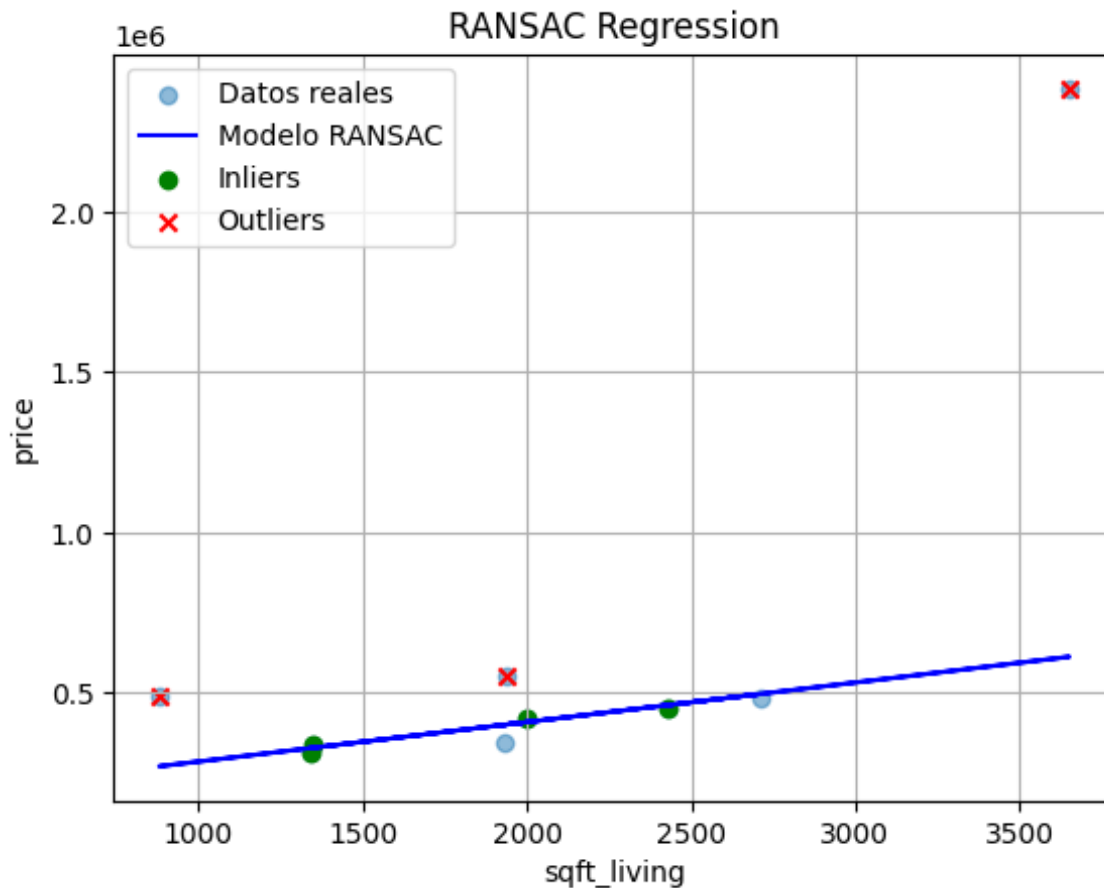


Figure 5: Regresión lineal RANSAC entre *sqft_living* y el precio de las viviendas.

La Figura 5 muestra la regresión obtenida mediante RANSAC, la cual se ajusta mejor a la tendencia general de los datos, identificando y excluyendo de forma automática los valores atípicos.

Actividad 5. Métricas de Evaluación

Para evaluar el desempeño del modelo de regresión se emplearon distintas métricas estadísticas, las cuales permiten analizar tanto el ajuste general como la significancia del modelo.

Pesos del modelo

Los coeficientes estimados del modelo fueron:

$$\mathbf{w} = \begin{bmatrix} -2.43292331 \times 10^{-15} \\ 0.759920248 \\ -0.266883193 \end{bmatrix}$$

Coeficiente de determinación (R^2)

El coeficiente de determinación R^2 indica qué proporción de la varianza en la variable dependiente es explicada por el modelo. Se calcula como:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Donde:

- y_i son los valores reales,
- $\hat{y}_i$ son los valores predichos por el modelo,
- $\bar{y}$ es el promedio de los valores reales.

En nuestro caso, se obtuvo:

$$R^2 = 0.7486$$

Lo que indica que el modelo explica aproximadamente el 74.86% de la variabilidad de los datos.

Estadístico F

El estadístico F evalúa si el modelo ajustado proporciona una mejora significativa sobre un modelo sin predictores. Se define como:

$$F = \frac{(SSR/p)}{(SSE/(n-p-1))}$$

Donde:

- SSR es la suma de cuadrados explicada (modelo),
- SSE es la suma de errores al cuadrado,
- p es el número de predictores,
- n es el número total de observaciones.

En esta actividad se obtuvo:

$$F = 98.5$$

Este valor sugiere que el modelo es estadísticamente significativo y que al menos una de las variables independientes tiene efecto sobre la variable dependiente.

Estas métricas permiten concluir que el modelo presenta un buen rendimiento predictivo y es adecuado para inferencias en esta base de datos.

Actividad 6. Regresión Multilineal

En esta actividad se implementó un modelo de regresión multilineal para predecir el precio de las viviendas utilizando múltiples variables predictoras. A diferencia de la regresión lineal simple, este modelo considera simultáneamente varios factores que pueden influir en el valor de la propiedad.

El modelo tiene la forma:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

Donde:

- $\hat{y}$ representa el precio estimado de la vivienda.
- $x_1, x_2, \dots, x_n$ son las variables independientes (por ejemplo, índice de pobreza, tasa de impuesto, etc.).
- w_0 es el sesgo (intersección), y w_i los coeficientes de cada variable.

Evaluación del modelo

Para evaluar el rendimiento del modelo se utilizó el coeficiente de determinación R^2 , que cuantifica qué tan bien los datos se ajustan al modelo:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

En este caso, se obtuvo:

$$R^2 = 0.6716$$

Esto indica que el modelo explica aproximadamente el 67.16% de la variabilidad en los precios de las viviendas, lo que sugiere un ajuste razonable, aunque mejorable con variables adicionales o transformaciones.

Predicción puntual

El modelo fue utilizado para estimar el precio de una vivienda con las siguientes características:

- Índice de pobreza: $x_1 = 0.1$

- Tasa de impuesto: $x_2 = 300$

El valor predicho por el modelo fue:

$$\hat{y} = 16,980.74$$

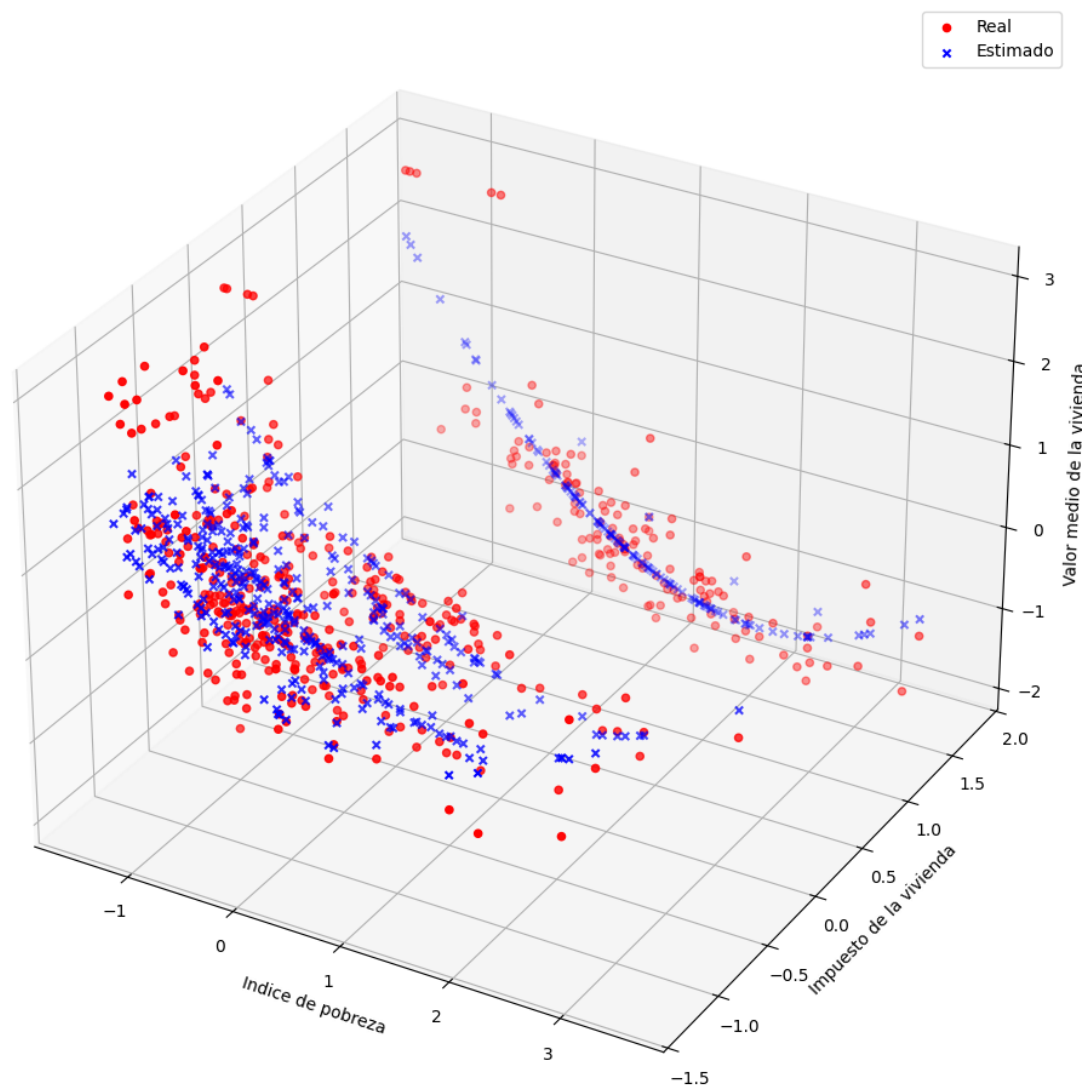


Figure 6: Superficie de costo y camino del gradiente descendiente

Actividad 7. Clasificación Softmax

La regresión Softmax es un modelo de clasificación multiclase que generaliza la regresión logística. Es especialmente útil cuando se desea clasificar datos en más de dos categorías. La función principal que define este modelo es la función Softmax, la cual convierte un vector de puntuaciones (logits) en probabilidades que suman 1.

Dada una entrada $\mathbf{x}$ y un conjunto de parámetros $\mathbf{W}$ y $\mathbf{b}$, el modelo calcula:

$$z_j = \mathbf{w}_j^\top \mathbf{x} + b_j$$

La función Softmax transforma estos valores z_j en probabilidades p_j mediante:

$$p_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}, \quad \text{para } j = 1, \dots, K$$

Donde K es el número de clases. El modelo se entrena minimizando la pérdida logarítmica o *cross-entropy*:

$$\mathcal{L} = - \sum_{i=1}^n \sum_{j=1}^K y_{ij} \log(p_{ij})$$

Donde y_{ij} es una variable indicadora que vale 1 si la clase verdadera del ejemplo i es la clase j , y 0 en caso contrario. El entrenamiento se realiza comúnmente mediante descenso del gradiente, ajustando los pesos para minimizar esta función de pérdida.

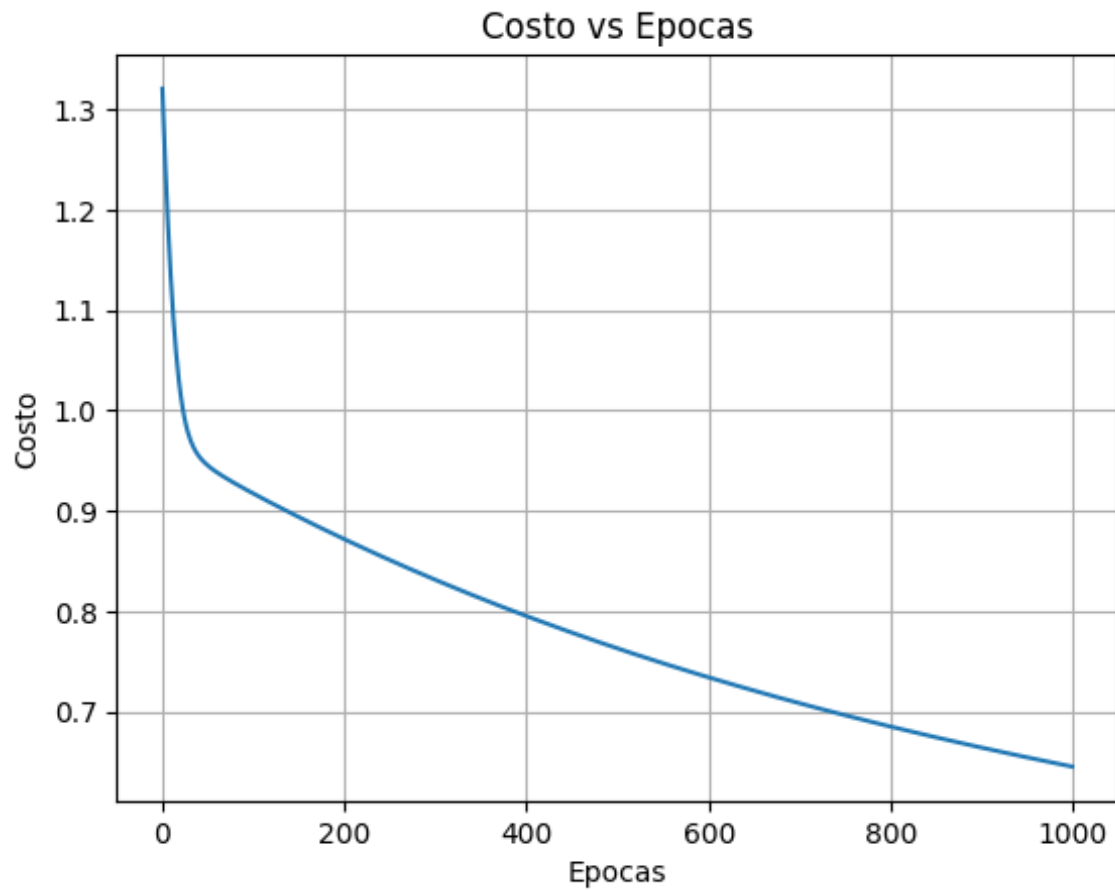


Figure 7: Superficie de costo y trayectoria del descenso por gradiente

Para evaluar el desempeño del modelo, se utilizó una matriz de confusión, la cual fue visualizada mediante un *heatmap* (mapa de calor), permitiendo observar gráficamente las predicciones correctas e incorrectas en cada clase.

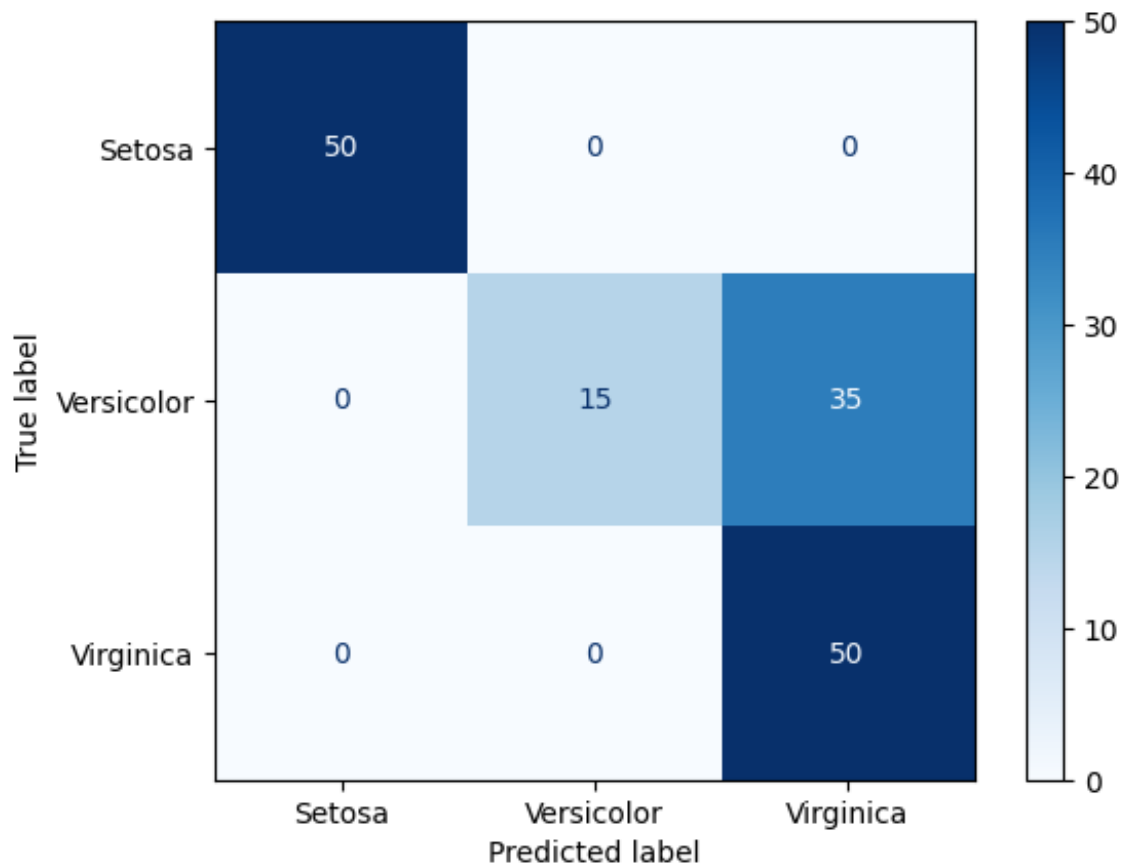


Figure 8: Matriz de confusión visualizada con un *heatmap* (mapa de calor)

Además, se generó una curva ROC (Receiver Operating Characteristic), la cual muestra la relación entre la tasa de verdaderos positivos (TPR) y la tasa de falsos positivos (FPR) para diferentes umbrales de decisión. En el contexto multiclase, se construyen curvas ROC para cada clase (uno contra el resto), y se pueden calcular métricas como el AUC (Área Bajo la Curva) para evaluar la capacidad del modelo de distinguir entre clases.

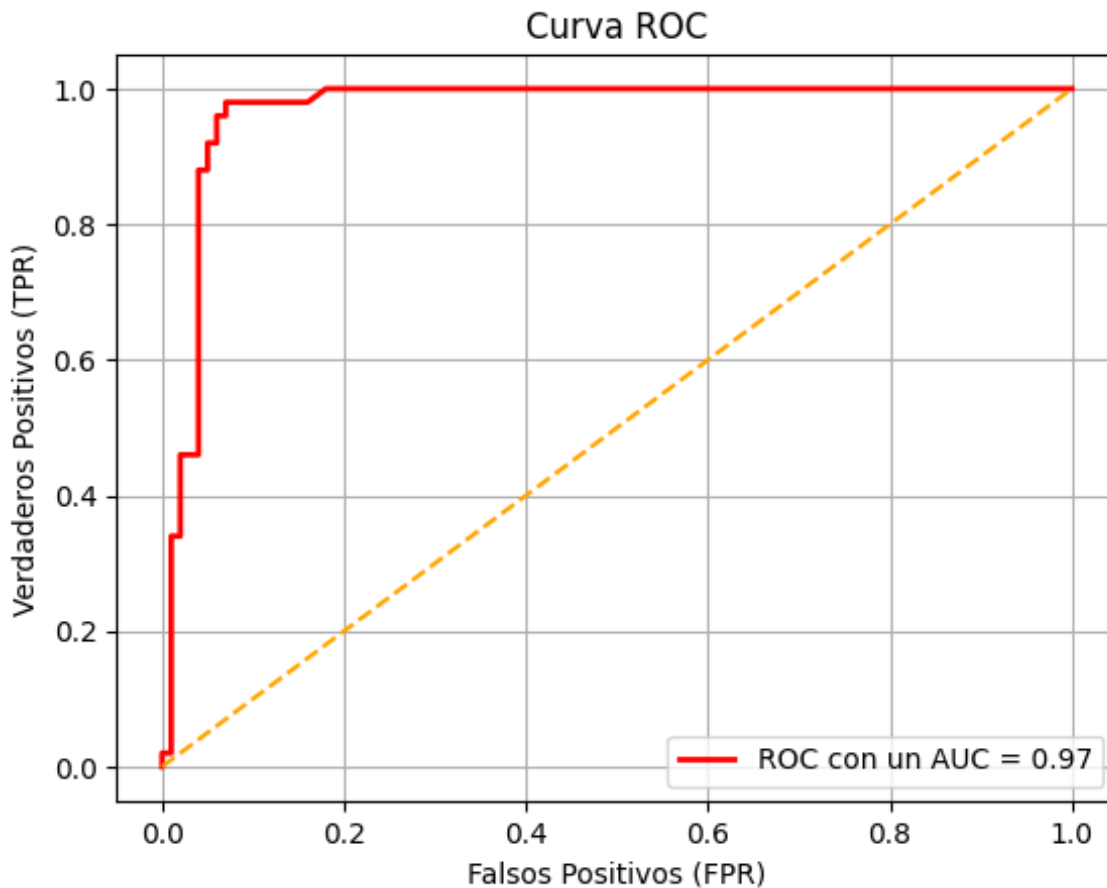


Figure 9: Curva ROC para la clasificación multiclase. Se observa la capacidad del modelo para distinguir entre clases individuales.

Actividad 8. Derivada de la Función de Costo de Softmax y Entropía Cruzada

La entropía cruzada para un modelo Softmax es:

$$H(p, q) = \sum_x p(x) \log((XW))$$

Ahora definimos la función de costo, teniendo en cuenta que la optimización se consistirá en encontrar el W que mejor separe las clases.

$$J(W) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c})$$

Donde:

- N es el número de datos.
- C es el número de clases.
- $y_{i,c}$ es la etiqueta en codificación one hot de dato i y la clase c .
- $p_{i,c}$ es la predicción de que el dato i pertenezca a la clase c , que se obtiene mediante la función softmax, es una probabilidad.

La probabilidad predicha con Softmax es:

$$p_{i,c} = \frac{e^{z_{i,c}}}{\sum_{c'=1}^C e^{z_{i,c'}}}$$

donde $z_{i,c} = W_k x_i + b_k$

Para obtener la derivada de la función de costo con respecto a los pesos, podemos utilizar la regla de la cadena:

$$\frac{\partial J(W)}{\partial W_k} = \sum_{i=1}^N \frac{\partial J(W)}{\partial z_{i,c}} \cdot \frac{\partial z_{i,c}}{\partial W_k}$$

Es evidente que $\frac{\partial z_{i,c}}{\partial W_k} = x_i$

Ahora para calcular la derivada de la función de Costo con respecto a $z_{i,c}$ volvemos a utilizar la regla de la cadena, la derivada de $J(W)$ con respecto a $z_{i,c}$ es:

$$\frac{\partial J(W)}{\partial z_{i,c}} = \sum_{c'=1}^C \frac{\partial J(W)}{\partial p_{i,c'}} \cdot \frac{\partial p_{i,c'}}{\partial z_{i,c}}$$

La derivada de la función de costo con respecto a $p_{i,c}$ es:

$$\frac{\partial J(W)}{\partial p_{i,c}} = -\frac{y_{i,c}}{p_{i,c}}$$

Por otro lado, la derivada de $p_{i,c}$ con respecto a $z_{i,c}$ es:

$$\frac{\partial p_{i,c}}{\partial z_{i,c'}} = p_{i,c}(\delta_{c,c'} - p_{i,c'})$$

donde $\delta_{c,c'}$ es el delta de Kronecker, que es 1 cuando $c = c'$ y 0 en caso contrario.

Sustituyendo las derivadas anteriores:

$$\frac{\partial J(W)}{\partial z_{i,c}} = \sum_{c'=1}^C \left(-\frac{y_{i,c'}}{p_{i,c'}} \right) \cdot p_{i,c'}(\delta_{c,c'} - p_{i,c'})$$

Este término se simplifica y se reduce a:

$$\frac{\partial J(W)}{\partial z_{i,c}} = p_{i,c} - y_{i,c}$$

Finalmente sustituimos los resultados en la primer regla de la cadena:

$$\frac{\partial J(W)}{\partial W_k} = \sum_{i=1}^N (p_{i,c} - y_{i,c}) \cdot x_i$$

Actividad 9. Clasificación de la Iris-Setosa con Naive Bayes

El clasificador **Naive Bayes** se basa en el **Teorema de Bayes**, suponiendo que los atributos son condicionalmente independientes dados los valores de clase.

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Dado que $P(X)$ es constante entre clases, se maximiza:

$$P(C) \cdot \prod_{i=1}^n P(x_i|C)$$

Se asume que cada atributo sigue una distribución **normal**, y se estima su media (o mediana como alternativa robusta) y varianza por clase. La probabilidad condicional se calcula como:

$$P(x_i|C) = \frac{1}{\sqrt{2\pi\sigma_C^2}} \cdot \exp\left(-\frac{(x_i - \mu_C)^2}{2\sigma_C^2}\right)$$

Para clasificar una nueva muestra, se calcula la probabilidad total para cada clase y se selecciona la clase con mayor probabilidad:

$$\hat{y} = \arg \max_C \prod_{i=1}^n P(x_i|C)$$

Evaluación del Modelo

La evaluación del modelo se realiza mediante la **precisión**, comparando las predicciones del clasificador con las etiquetas reales del conjunto de prueba:

$$\text{Precisión} = \frac{\text{Número de aciertos}}{\text{Número total de muestras}}$$

Este valor resume qué tan bien el modelo clasifica las muestras del conjunto de prueba.

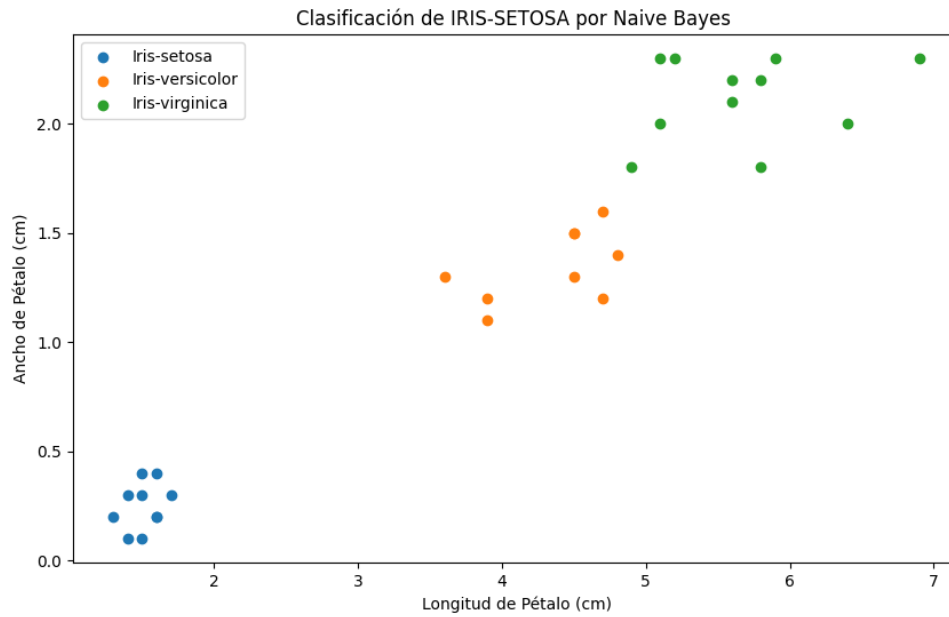


Figure 10: Clasificación de Iris setosa con Naive Bayes (Visualizacion datos de prueba).

Actividad 10. Clasificación de datos Dummy mediante SVM

El método de *Support Vector Machines* (SVM) es una técnica de aprendizaje supervisado utilizada para clasificación y regresión. En esta actividad, se aplicó SVM para clasificar un conjunto de datos dummy linealmente separables. El objetivo de SVM es encontrar el hiperplano que maximiza el margen entre las dos clases de datos. Matemáticamente, este hiperplano se define como:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

Donde:

- $\mathbf{w}$ es el vector normal al hiperplano.
- b es el sesgo o término independiente.
- $\mathbf{x}$ es el vector de características de entrada.

La función de decisión que determina la clase de un nuevo punto es:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

Para maximizar el margen, se resuelve el siguiente problema de optimización:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{sujeto a} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad \forall i$$

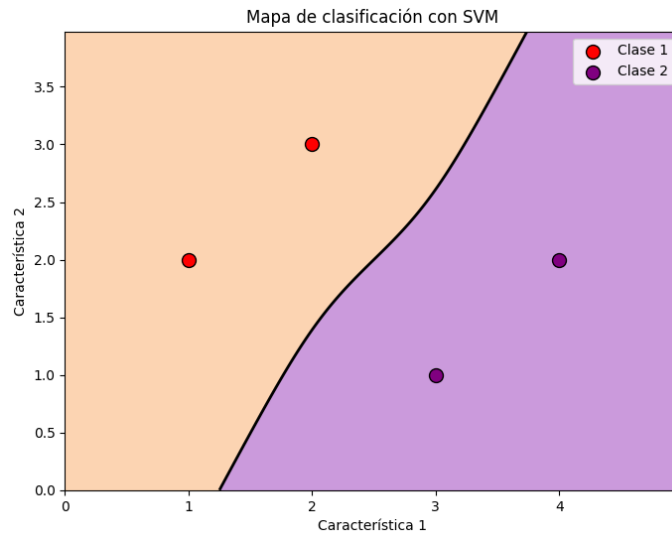


Figure 11: SVM con datos Dummy

Actividad 11. Clasificación de datos random XOR mediante SVM

En esta actividad se aplicó SVM a un conjunto de datos con una distribución tipo XOR, que no es linealmente separable. En estos casos, se utiliza el método del *kernel trick*, que transforma los datos a un espacio de mayor dimensión donde sí es posible separar las clases linealmente. Una función kernel común es el kernel Gaussiano o RBF:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

El problema de optimización en el espacio transformado se convierte en:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \\ \text{sujeto a} \quad & \sum_{i=1}^n \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C \end{aligned}$$

Donde C es un parámetro de penalización y α_i son los multiplicadores de Lagrange. El uso de un kernel permite que SVM se adapte a patrones no lineales complejos como el XOR.

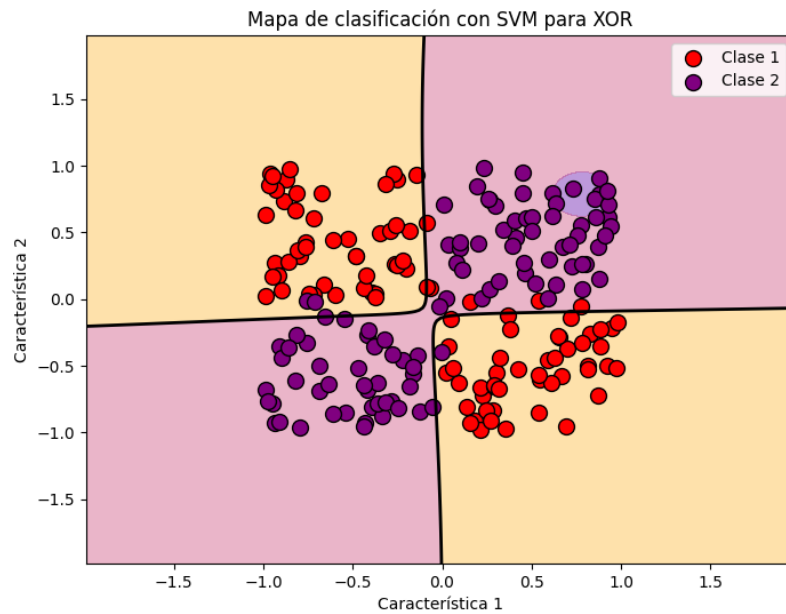


Figure 12: SVM con datos XOR

Actividad 12. Clasificación de Iris-setosa mediante KNN

En esta actividad se utilizó k-Nearest Neighbors (k-NN) el cual es un algoritmo de aprendizaje supervisado utilizado tanto para clasificación como regresión. Clasifica un nuevo punto en base a la mayoría de las clases de sus k vecinos más cercanos.

Se parte de un conjunto de entrenamiento:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n \quad \text{con } \mathbf{x}_i \in \mathbb{R}^d, y_i \in \mathcal{C}$$

donde $\mathcal{C}$ es el conjunto de clases posibles (en clasificación).

Proceso de Clasificación

1. Calcular la distancia entre $\mathbf{x}$ y cada $\mathbf{x}_i$ del conjunto de entrenamiento. Para un punto nuevo $\mathbf{x}$, se calcula su distancia a cada punto del entrenamiento:

$$d(\mathbf{x}, \mathbf{x}_i) = \|\mathbf{x} - \mathbf{x}_i\|_2 = \sqrt{\sum_{j=1}^d (x_j - x_{ij})^2}$$

2. Seleccionar los k puntos más cercanos según la distancia.
3. Asignar a $\mathbf{x}$ la clase más común entre esos k vecinos:

$$\hat{y} = \text{mode}(\{y_{(i)}\}_{i=1}^k)$$

donde $y_{(i)}$ son las etiquetas de los k vecinos más cercanos.

Evaluación del modelo

La métrica de evaluación más común en clasificación:

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(\hat{y}_i = y_i)$$

donde $\mathbb{I}$ es la función indicadora.

A continuación se puede observar el resultado de la clasificación, es importante notar que se logró una precisión del 100

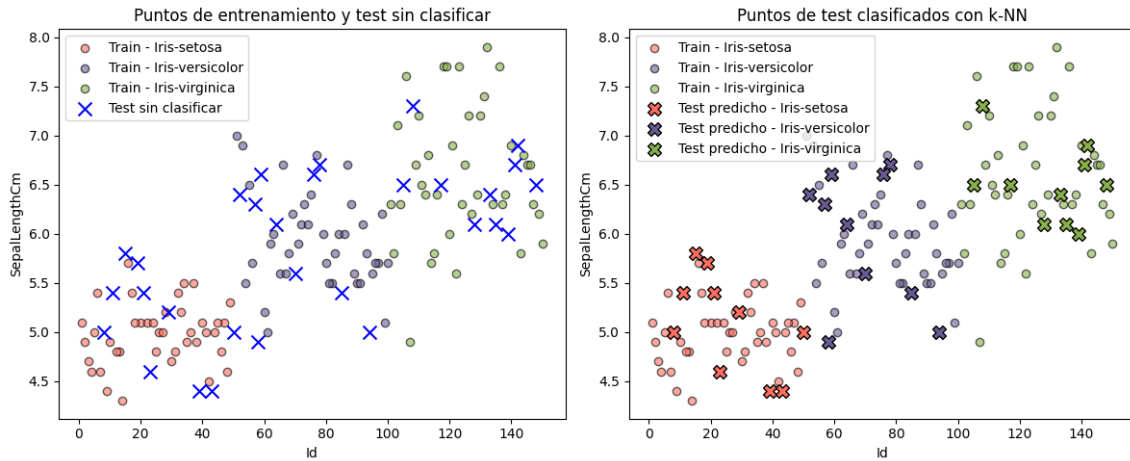


Figure 13: Clasificación de mediante KNN.

Resultados de Ejemplo

- Tiempo total de ejecución: **0.0165 segundos**
- Precisión del clasificador k-NN (k=3) sobre el conjunto de prueba: **100.00%**

Actividad 13. Reduccion de dimensionalidad mediante PCA y clasificacion de Iris-setosa mediante KNN

En esta actividad reconocemos las ventajas de aplicar una reducción de dimensionalidad mediante PCA (Principal Component Analysis) cuando vamos a realizar una clasificación, para ello aplicamos un PCA por el método de covarianza.

Procedimiento de PCA por el metodo de Covarianza

Dado un conjunto de datos $\mathbf{X} \in \mathbb{R}^{n \times d}$:

1. **Centrado de los datos:**

$$\mathbf{D} = \mathbf{X} - \bar{\mathbf{X}}, \quad \text{donde } \bar{\mathbf{X}} \text{ es el vector promedio}$$

2. **Matriz de covarianza:**

$$\mathbf{S} = \frac{1}{n-1} \mathbf{D}^T \mathbf{D}$$

3. **Cálculo de valores y vectores propios:**

$$\mathbf{S} \mathbf{v}_i = \lambda_i \mathbf{v}_i$$

Donde λ_i son los valores propios (varianza explicada) y $\mathbf{v}_i$ los vectores propios (direcciones principales).

4. Ordenar y seleccionar k componentes principales:

$$\mathbf{W} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k]$$

5. Proyección de los datos al nuevo espacio:

$$\mathbf{Y} = \mathbf{D}\mathbf{W}$$

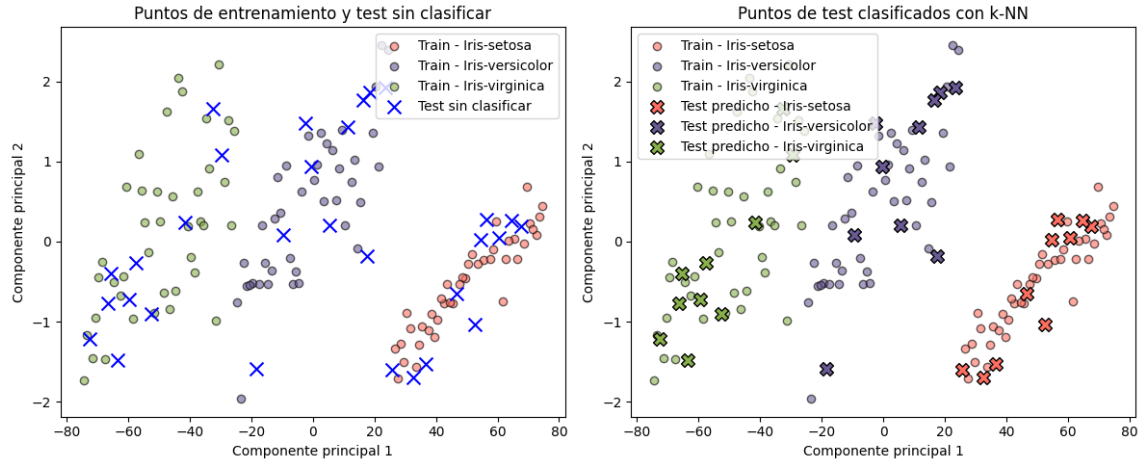


Figure 14: Clasificación de mediante KNN a datos previamente tratados con PCA.

Resultados de Ejemplo

- Tiempo total de ejecución: **0.0039 segundos**
- Precisión del clasificador k-NN (k=3) sobre el conjunto de prueba: **100.00%**

Cabe destacar que la visualización de la información mejoró considerablemente si comparamos la Figura 13 y la Figura 14, en esta última, se observa una mejor agrupación de la información. Adicionalmente, el tiempo de ejecución disminuyó considerablemente para el proceso con PCA.

Actividad 14. Fuzzy C Means

Fuzzy C-Means (FCM) es un algoritmo de clustering difuso que permite que un mismo dato pertenezca a varios clústeres con distintos grados de pertenencia. A diferencia del algoritmo K-means clásico (clustering duro), FCM asigna valores de pertenencia en lugar de pertenencia absoluta.

Función objetivo

El algoritmo minimiza la siguiente función de costo:

$$J_m = \sum_{i=1}^N \sum_{j=1}^c u_{ij}^m \cdot \|x_i - v_j\|^2$$

Donde:

- N : número total de datos.
- c : número de clústeres.
- $x_i \in \mathbb{R}^n$: el i -ésimo dato.
- $v_j \in \mathbb{R}^n$: centro del clúster j .
- $u_{ij} \in [0, 1]$: grado de pertenencia de x_i al clúster j .
- $m > 1$: parámetro de difusividad (típicamente $m = 2$).

Algoritmo iterativo

1. Actualización de los centros de clúster

$$v_j = \frac{\sum_{i=1}^N u_{ij}^m \cdot x_i}{\sum_{i=1}^N u_{ij}^m}$$

2. Actualización de los grados de pertenencia

$$u_{ij} = \frac{1}{\sum_{k=1}^c \left(\frac{\|x_i - v_j\|}{\|x_i - v_k\|} \right)^{\frac{2}{m-1}}}$$

3. Criterio de parada

El algoritmo se detiene cuando el cambio entre matrices de pertenencia sucesivas es menor que una **tolerancia** $\varepsilon > 0$:

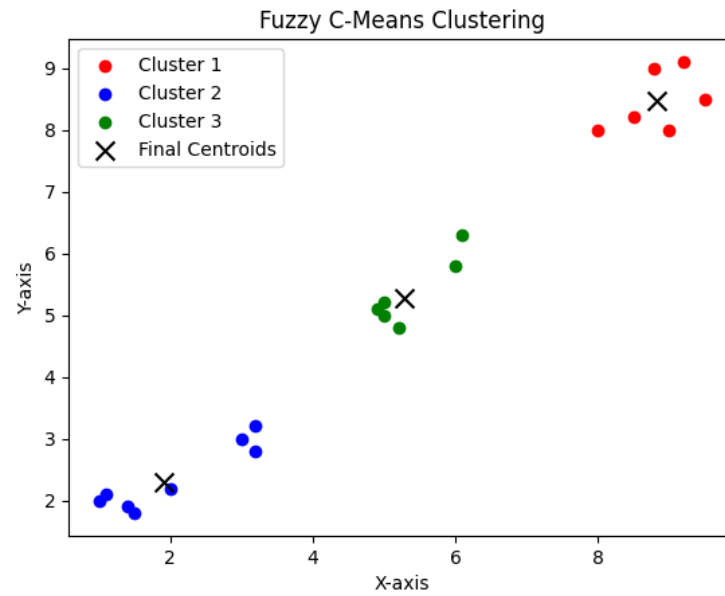
$$\max_{i,j} |u_{ij}^{(t+1)} - u_{ij}^{(t)}| < \varepsilon$$

o cuando se alcanza el número máximo de iteraciones predefinido.

En esta actividad se utilizó el algoritmo Fuzzy C-Means (FCM) para clasificar un conjunto de datos sintéticos. En la Figura 16.a) se muestra la posición inicial de los centroides, la cual no permite una diferenciación clara entre las clases. Posteriormente, se ejecuta el algoritmo FCM, el cual ajusta iterativamente la posición de los centroides con base en la pertenencia difusa de los datos. En la Figura 16.b) se observa la disposición final de los centroides y la estructura de agrupamiento resultante, donde los datos han sido correctamente organizados en clústeres de acuerdo con su similitud.



(a) Clustering Inicial



(b) Clustering Final

Figure 15: Proceso de clustering Fuzzy C-Means: desde la inicialización hasta la convergencia.

Adicionalmente, se presenta la gráfica que muestra la convergencia del cambio en la posición de los centroides. Estos cambios se evalúan en cada iteración y se comparan contra un umbral de

tolerancia. El algoritmo se detiene cuando la variación entre las posiciones de los centroides en iteraciones consecutivas es suficientemente pequeña, es decir, cuando dicho cambio es menor que la tolerancia establecida. Esto indica que el algoritmo ha alcanzado un estado de estabilidad o convergencia.

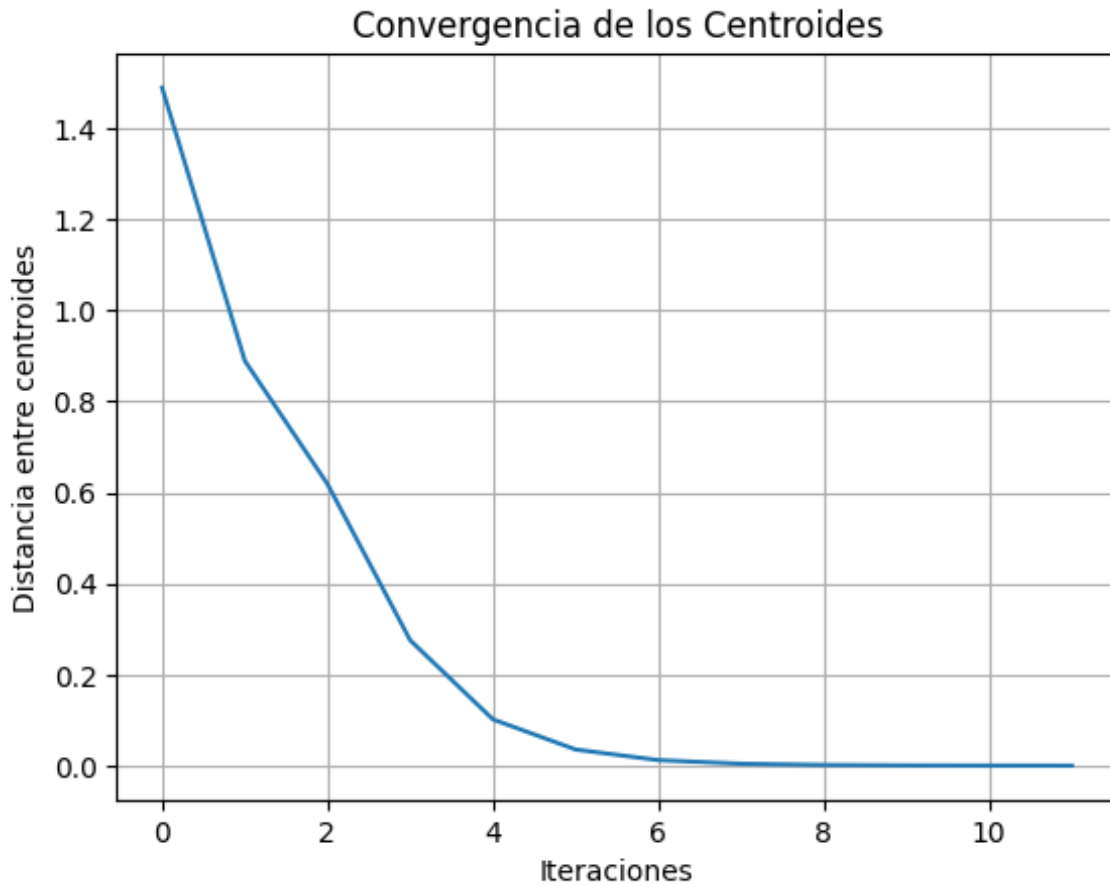


Figure 16: Convergencia de los centroides.

Actividad 16. Red con datos XOR

Esta actividad consiste en entrenar una red neuronal multicapa (perceptrón multicapa o MLP) para aproximar una función lógica no lineal, en este caso, la función XOR. Este entrenamiento se realiza a través del algoritmo de backpropagation con descenso del gradiente, utilizando datos de entrada y sus respectivas etiquetas, bajo el paradigma del aprendizaje supervisado.

1. Arquitectura

Una red neuronal con:

- 2 neuronas de entrada (para x_1 y x_2)
- 1 capa oculta con 2 neuronas
- 1 neurona de salida
- Función de activación: **sigmoide**

2. Funciones de activación

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

3. Propagación hacia adelante (Forward Propagation)

$$\begin{aligned} z^{(1)} &= x \cdot w^{(1)} + b^{(1)} \\ h &= \sigma(z^{(1)}) \\ z^{(2)} &= h \cdot w^{(2)} + b^{(2)} \\ \hat{y} &= \sigma(z^{(2)}) \end{aligned}$$

4. Función de pérdida

Función de error (entropía cruzada simplificada aquí como error cuadrático medio):

$$\mathcal{L} = \frac{1}{2}(y - \hat{y})^2, \quad \frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y$$

5. Backpropagation

Capa de salida

$$\begin{aligned} \delta^{(2)} &= \hat{y} - y \\ \frac{\partial \mathcal{L}}{\partial w^{(2)}} &= h^T \cdot \delta^{(2)} \\ \frac{\partial \mathcal{L}}{\partial b^{(2)}} &= \sum \delta^{(2)} \end{aligned}$$

Capa oculta

$$\begin{aligned} \delta^{(1)} &= (\delta^{(2)} \cdot (w^{(2)})^T) \circ \sigma'(z^{(1)}) \\ \frac{\partial \mathcal{L}}{\partial w^{(1)}} &= x^T \cdot \delta^{(1)} \\ \frac{\partial \mathcal{L}}{\partial b^{(1)}} &= \sum \delta^{(1)} \end{aligned}$$

6. Actualización de pesos

$$w := w - \alpha \cdot \frac{\partial \mathcal{L}}{\partial w}, \quad b := b - \alpha \cdot \frac{\partial \mathcal{L}}{\partial b}$$

A continuación se muestran los resultados de la red neuronal entrenada para resolver el problema lógico XOR:

- Salida original (y):

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

- Salida predicha ($\hat{y}$):

$$\begin{bmatrix} 0.0 \\ 1.0 \\ 1.0 \\ 0.0 \end{bmatrix}$$

Como se observa, la salida predicha por la red neuronal coincide con la salida original, lo cual indica que el modelo ha aprendido correctamente la función XOR.